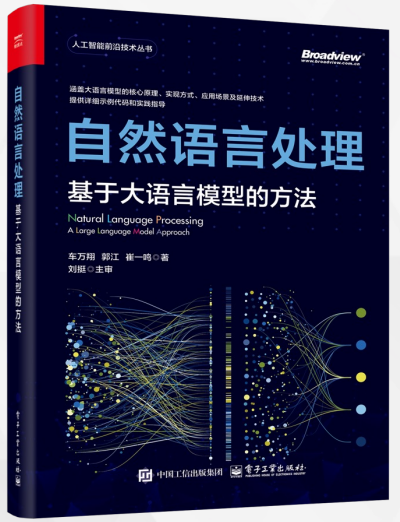



# 预训练词向量


**车万翔**

社会计算与交互机器人研究中心（HIT-SCIR）
哈尔滨工业大学

# 本讲内容与学习目标

| 课堂内容 | 学习目标 |
| --- | --- |
| 静态词向量训练 | 读懂CBOW、SGNS和GloVe及其联系 |
| 评价与迁移应用 | 正确检索、对齐词表和冻结参数 |
| 上下文词向量 | 理解字符CNN、BiLM与ELMo |
| 研究拓展与练习 | 比较表示粒度，了解前沿进展 |

# 词表示：独热编码与稠密向量

| 词 | 独热编码（节选） | 稠密向量（示意） |
| --- | --- | --- |
| 土豆 | [1, 0, 0, …] | [0.2, −0.4, 0.7, …] |
| 马铃薯 | [0, 1, 0, …] | [0.3, −0.3, 0.6, …] |
| 电脑 | [0, 0, 1, …] | [−0.6, 0.5, 0.1, …] |

**词表有V个词，每个词用D维向量表示，通常D远小于V。**

# Embedding层就是按ID查表

| 输入与参数 | 形状 | 示例 |
| --- | --- | --- |
| 词元ID | [B, T] | [[2, 3, 4, 5, 6]] |
| 向量表 E | [V, D] | E[4]是“吃”的向量 |
| 查询结果 E[ids] | [B, T, D] | 每个词元位置得到D维向量 |

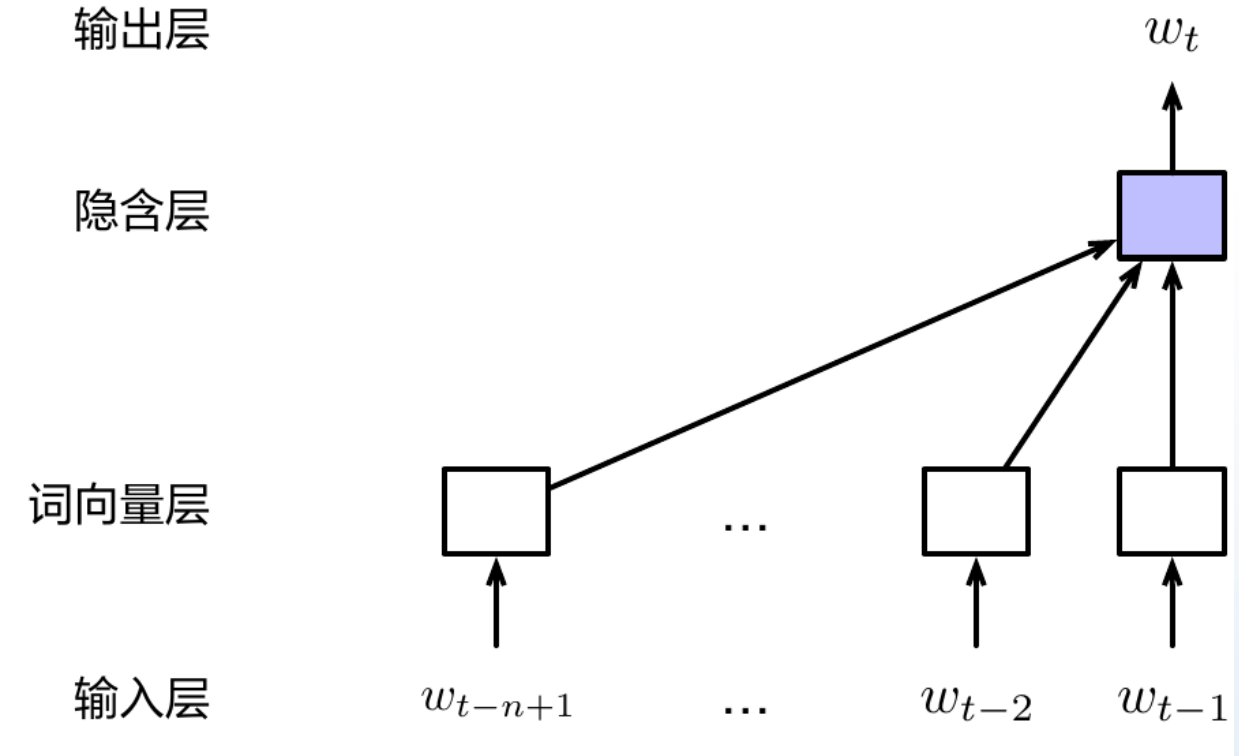
**同一个词ID在同一张静态词表中，总是查询到同一个向量。**

# 语言模型也能学到词向量

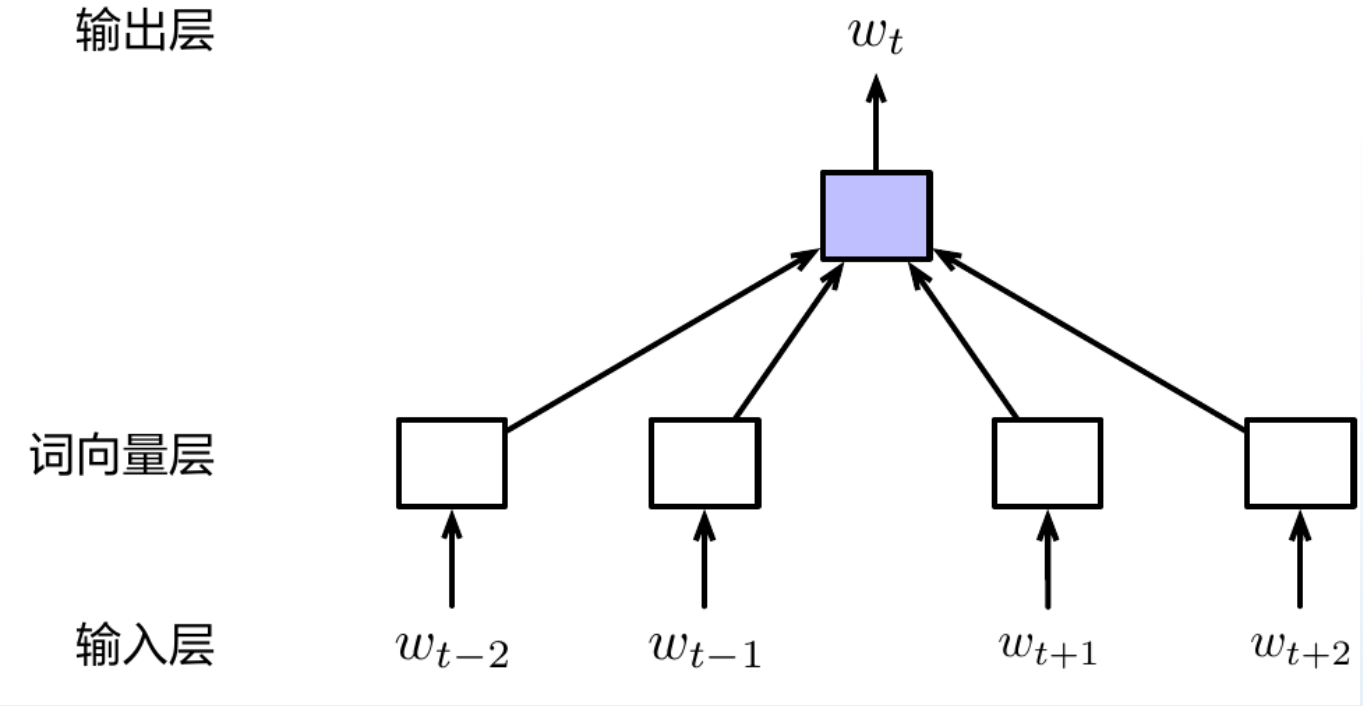
**预测目标** 利用前文预测下一词

**参数更新** 误差同时更新向量表

**知识迁移** 将向量用于新任务



# CBOW：利用周围词预测中心词




输入
中心词左右的上下文

输出
中心词在词表中的概率

**上下文求平均**
**因此不保留窗口内词序**

# CBOW训练样本与平均池化

我　喜欢　吃　新鲜　苹果

| 上下文词 | 我 | 喜欢 | 新鲜 | 苹果 |
| --- | --- | --- | --- | --- |
| 教学向量 | [1, 0] | [0, 1] | [1, 1] | [2, 0] |

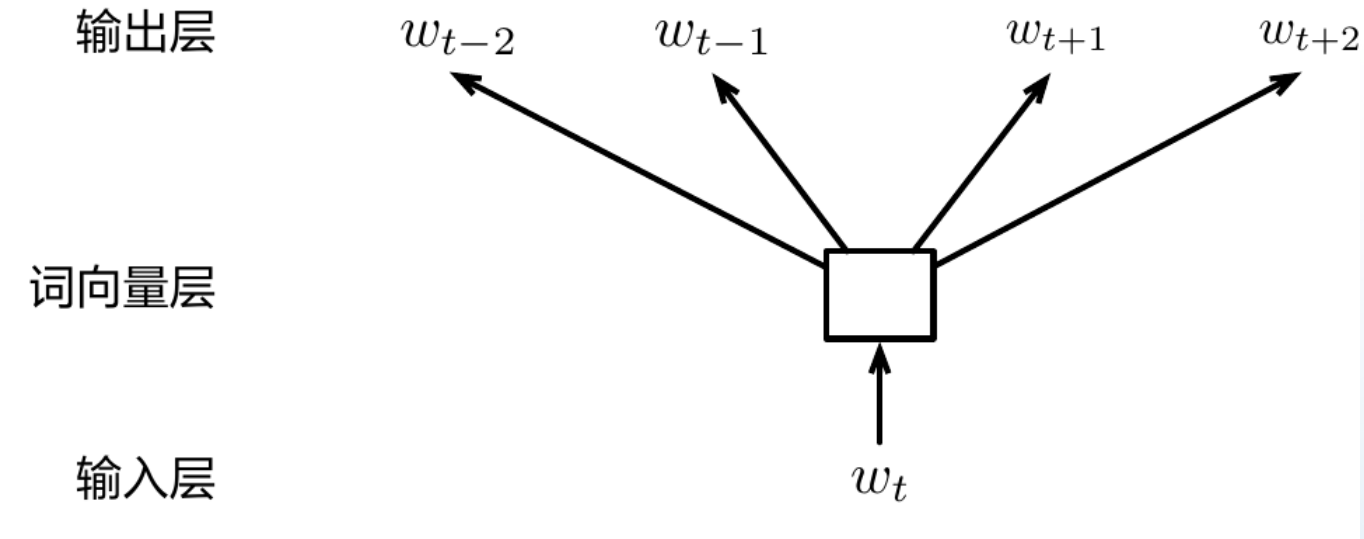
平均向量 h = [1, 0.5]，训练目标为“吃”。

# PyTorch实现：CBOW核心计算

```
import torch
from torch import nn
import torch.nn.functional as F
emb = nn.Embedding(8, 4)
head = nn.Linear(4, 8, bias=False)
ctx = torch.tensor([[2, 3, 5, 6], [2, 3, 5, 7]])
target = torch.tensor([4, 4])
h = emb(ctx).mean(dim=1)            # [2, 4]
logits = head(h)                    # [2, 8]
loss = F.cross_entropy(logits, target)
```

本例上下文长度相同，均不含PAD。loss.backward()计算梯度，优化器更新参数。

# Skip-gram：利用中心词预测周围词




输入
一个中心词

输出
窗口内的上下文词

**每个共现词对**
**贡献一项训练损失**

# Skip-gram的训练词对

| 中心词 w | 上下文 c | 一个预测事件 |
| --- | --- | --- |
| 吃 | 我 | P(我 \| 吃) |
| 吃 | 喜欢 | P(喜欢 \| 吃) |
| 吃 | 新鲜 | P(新鲜 \| 吃) |
| 吃 | 苹果 | P(苹果 \| 吃) |

**窗口越大，包含的关系越广，也会引入更远的共现噪声。**

# 完整Softmax与两张向量表

$$score(w, c) = u_c^T v_w$$

$$P(c \mid w) = \exp(score(w, c)) / \Sigma_j \exp(score(w, j))$$

| 参数 | 形状 | 作用 |
| --- | --- | --- |
| 输入向量表 E | V × D | 查询中心词向量v |
| 输出向量表 U | V × D | 查询候选上下文向量u |

# 词向量训练目标与计算量

| 训练方式 | 每个词对的主要代价 | 输出的含义 |
| --- | --- | --- |
| 完整Softmax | O(VD) | 归一化的上下文概率 |
| 层次Softmax | 常见平衡树为O(D log V) | 沿树路径分解概率 |
| 负采样 SGNS | O((K+1)D) | 区分数据词对与噪声词对 |

**K通常远小于V。负采样改变了目标函数。**

SGNS = Skip-gram with Negative Sampling。K为每个正词对抽取的负样本数。

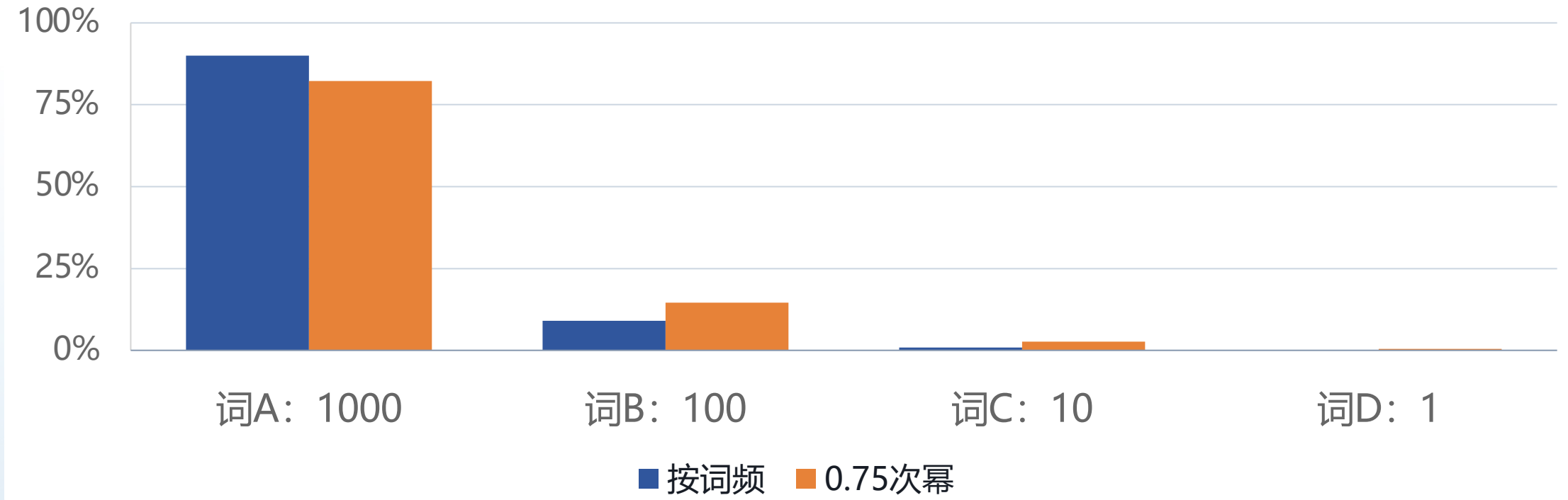
# 负样本：由噪声分布构造的词对

| 中心词 | 候选上下文 | 来源标签 |
| --- | --- | --- |
| 吃 | 苹果 | 1：语料窗口中的词对 |
| 吃 | 电脑 | 0：本次噪声采样 |
| 吃 | 栽种 | 0：本次噪声采样 |

**负样本标签描述本次训练样本的来源。**

# 负采样分布：压低高频词的优势

$$q(c) \propto \text{frequency}(c)^{0.75}$$




教学词频。高频词仍更常被抽中，但分布比原始词频更平缓。

# SGNS损失：拉高正词对，压低噪声词对

$$L = -\log \sigma(u_c^T v_w) - \Sigma_i \log \sigma(-u_{ni}^T v_w)$$

| 部分 | 希望的变化 | 理由 |
| --- | --- | --- |
| 正词对分数 a | 增大 | σ(a)更接近1 |
| 负词对分数 b | 减小 | σ(−b)更接近1 |

**若各分数都为0，K = 2，则 L = 3 ln 2 ≈ 2.079。**

σ是Sigmoid函数。实际代码用logsigmoid计算，避免先取概率再取对数。

# PyTorch实现：正负词对的损失

```
vin, vout = nn.Embedding(8, 4), nn.Embedding(8, 4)
center = torch.tensor([4, 4])
pos_ids = torch.tensor([6, 7])
neg_ids = torch.tensor([[2, 3], [2, 5]])
v = vin(center)
pos = (v * vout(pos_ids)).sum(-1)       # [B]
neg = (v[:, None, :] * vout(neg_ids)).sum(-1)
loss = -(F.logsigmoid(pos) +
         F.logsigmoid(-neg).sum(-1)).mean()
loss.backward()
```

neg的形状为[B, K]。实际训练排除PAD，并明确噪声采样策略。

# GloVe：显式利用全局共现统计

## 中心词“吃”，窗口半径为2

| 上下文 | 我 | 喜欢 | 新鲜 | 苹果 |
| --- | --- | --- | --- | --- |
| 距离 d | 2 | 1 | 1 | 2 |
| 本次计数增量 1/d | 0.5 | 1.0 | 1.0 | 0.5 |

## 遍历语料并累加，得到全局共现矩阵 M。

本例采用距离倒数权重。共现计数可以是小数，不必是整数。

# GloVe的加权回归目标

$$r(w, c) = v_w^T u_c + b_w + b_c - \ln M_{wc}$$

$$L = \Sigma\, f(M_{wc})\, r(w, c)^2 \quad (仅\ M_{wc} > 0)$$

$$f(x) = \min((x / 100)^{0.75}, 1)$$

| 共现次数 x | 1 | 10 | 100 | 1000 |
|---|---|---|---|---|
| 权重 f(x) | 0.032 | 0.178 | 1.000 | 1.000 |

只遍历M > 0的共现词对。偏置吸收部分词频效应，权重调节样本影响。

# PyTorch实现：GloVe的维度对齐

```
vin, vout = nn.Embedding(8, 4), nn.Embedding(8, 4)
bw, bc = nn.Embedding(8, 1), nn.Embedding(8, 1)
counts = torch.tensor([3.0, 5.0])
score = (vin(center) * vout(pos_ids)).sum(-1)
score = score + bw(center).squeeze(-1)
score = score + bc(pos_ids).squeeze(-1)   # [B]
weight = (counts / 100).pow(0.75).clamp(max=1)
loss = (weight * (score - counts.log()).square()).mean()
loss.backward()
```

偏置要压成[B]。计数必须大于0，每个词对恰好产生一个加权残差。

# 静态词向量方法比较

| 方法 | 学习依据 | 直接优化的对象 |
| --- | --- | --- |
| CBOW | 上下文平均表示 | 中心词预测 |
| Skip-gram + NS | 中心词与上下文词对 | 正负词对分类 |
| GloVe | 全局累计的共现矩阵 | 对数共现量回归 |

这些方法通常为每个词类型保存一个向量。语料和超参数同样影响最终效果。

# 统一视角：SGNS隐式分解PMI矩阵

最优时 $v_w^T u_c \approx PMI(w, c) - \ln K$

| | | | |
|---|---|---|---|
| **计数方法** | 共现计数 → PPMI矩阵 | 截断SVD | $W = U_D \Sigma_D$ |
| **预测方法** | 正词对 + K个负样本 | SGD优化SGNS损失 | $E U^T \approx PMI - \ln K$ |

**显式构造矩阵和逐对预测，刻画的是同一类统计量：词与上下文的关联强度。**

**训练动态** 在可解析的近似模型中，词向量按正交子空间逐个习得，有效秩分步增长（Karkada等，2025）

**类比来源** 类比结构已出现在P(w,c)/(P(w)P(c))矩阵的主特征向量中，取对数后更明显（Korchinski等，2025）

Levy & Goldberg，NeurIPS 2014；后两项均为NeurIPS 2025。等式需维度足够、充分训练，实际只是近似。

# 词向量训练中的关键选择

| | |
|---|---|
| **语料与切分** | 领域是否匹配，中文分词是否一致 |
| **窗口与维度** | 局部关系范围，以及表示容量和存储开销 |
| **采样与边界** | 高频词下采样、负样本数、PAD的排除 |
| **导出与复现** | 记录词表顺序、参数设置与随机种子 |

保存向量的同时保存词表。单独一张矩阵无法说明每一行对应哪个词。

# 词向量评价：余弦相似度

$$\cos(x, y) = (x^T y) / (||x||_2 \, ||y||_2)$$

| 向量 | 与x = [1, 0]的余弦 | 解释 |
| --- | --- | --- |
| y = [0.8, 0.6] | 0.8 | 方向较接近 |
| z = [0, 1] | 0 | 方向正交 |

# PyTorch实现：相似词检索

```
qid, k = 0, 2
norm = W.norm(dim=1)
assert norm[qid] > 0
valid = norm > 0
valid[qid] = False
assert k <= int(valid.sum())
unit = F.normalize(W, dim=1)
scores = unit @ unit[qid]
scores = scores.masked_fill(~valid, -torch.inf)
values, indices = scores.topk(k)
```

W为[V, D]词向量矩阵。把检索结果ID转换为词时，必须使用W对应的词表。

# 教材示例：近邻检索与人工相似度

## GloVe近邻（余弦相似度）

| 查询词 | 最相近的3个词 |
| --- | --- |
| august | september 0.832<br>july 0.803 · june 0.765 |
| good | **bad 0.730**<br>funny 0.692 · tough 0.685 |

## WordSim353人工评分（0–10）

| 词对 | 平均分 |
| --- | --- |
| money – cash | 9.15 |
| love – sex | 6.77 |
| lad – brother | 4.46 |
| development – issue | 3.97 |
| stock – jaguar | 0.92 |

**定量评价：计算模型余弦与人工评分之间的Spearman相关系数。**

**注意**　“good”的最近邻是“bad”：分布相似也会把反义词拉近，与“分布假设”页的局限一致

教材使用斯坦福GloVe向量。WordSim353含353个英文词对，每对由16位标注者打分取平均。

# 词类比：关系方向的近似迁移

**man : woman ≈ king : ?**

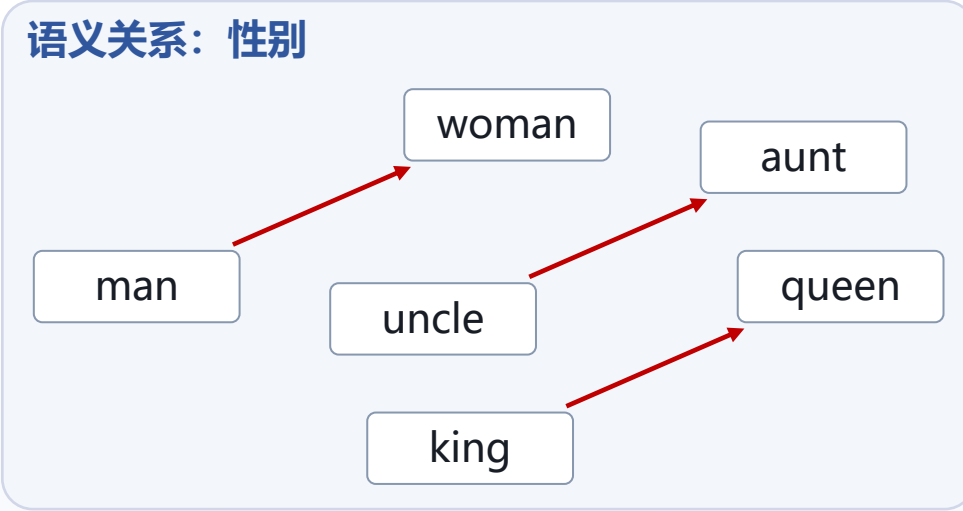
**q = v(king) + v(woman) – v(man)**

**预测 = argmax cos(q, v(w))**

w遍历词表中的候选词

**候选排除** man、woman、king，以及零向量

**结果解释** 理想示例为queen，实际结果取决于模型和语料







平行的箭头表示相同关系。向量类比是近似规律，不保证每一组关系都成立。

# 内在评价与下游任务评价

| 评价方式 | 观察内容 | 需要控制的条件 |
| --- | --- | --- |
| 相似度、类比 | 词级语义关系 | 词表覆盖率、候选与指标 |
| 下游任务 | 分类、标注等应用效果 | 相同数据划分与模型结构 |
| 二维可视化 | 局部聚类与异常点 | 降维方法和随机种子 |

**选词向量时，最终依据目标任务的验证集表现。**

相似词表好看，不意味着任务准确率一定更高。测试集只用于最终评价。

# 预训练词向量在下游任务中的使用

| 使用方式 | 词向量参数 | 适用考虑 |
| --- | --- | --- |
| 固定特征 | 冻结，仅训练任务模型 | 标注少、降低可训练参数量 |
| 参数初始化 | 随下游损失继续更新 | 允许表示适应任务与领域 |

**情感分类**　　按词查表，句子池化，再预测情感标签

**序列标注**　　保留每个位置的表示，再预测词性或实体标签

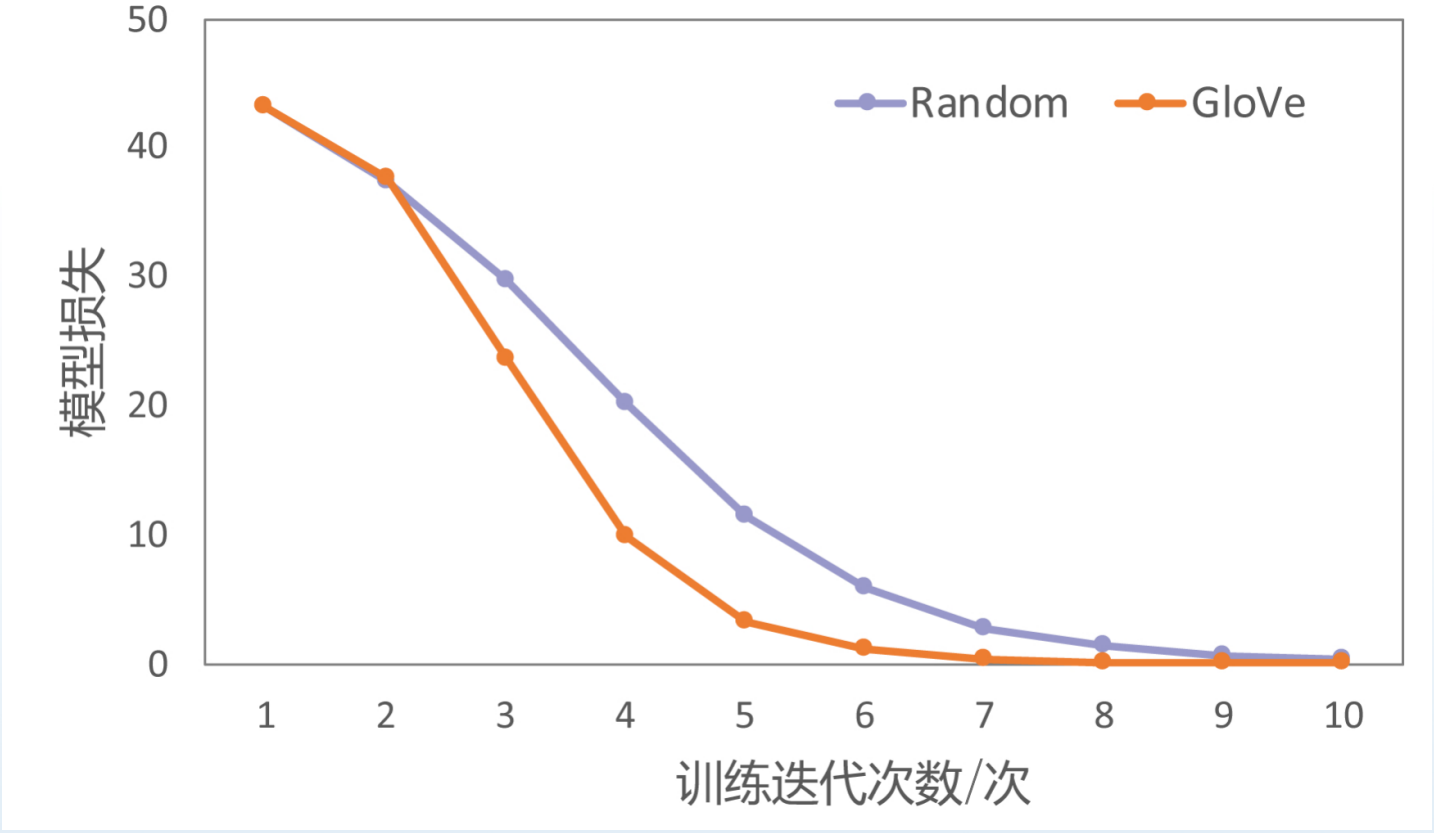
“静态”描述同一词的表示不随上下文变化。它与训练时是否冻结参数是两回事。

# PyTorch实现：冻结向量与有效词池化

```
emb = nn.Embedding.from_pretrained(
    W, freeze=True, padding_idx=0)
ids = torch.tensor([[2, 1, 0], [3, 2, 1]])
mask = ids.ne(0)
x = emb(ids)
pooled = (x * mask[..., None]).sum(dim=1)
pooled = pooled / mask.sum(1, keepdim=True).clamp_min(1)
classifier = nn.Linear(W.shape[1], 2)
logits = classifier(pooled)
```

将freeze改为False即可微调。平均池化分母只统计有效词，不包括PAD。

# 教材案例：情感分类中的预训练收益




**测试准确率**
**随机初始化：67%**
**GloVe初始化：70%**

**该实验中**
**收敛更快**

教材中的小规模实验。3个百分点的差距仍需通过重复实验判断稳定性。

# 一词多义与上下文词向量

| 句子 | “下场”的含义 |
| --- | --- |
| 他 亲自 下场 参加 比赛 | 动词：亲自参与 |
| 竟 落得 这样 的 下场 | 名词：结局 |

**静态向量：v(下场)**
**上下文向量：h(下场, 所在句子)**

上下文改变，输出表示随之改变。模型参数在普通推断过程中保持不变。

# 早期探索：用双语资源区分词义

| 中文语境 | 英语对应线索 | 义项 |
| --- | --- | --- |
| 他 穿着 制服 | uniform | 名词：服装 |
| 他 制服 了 对手 | subdue | 动词：制伏 |

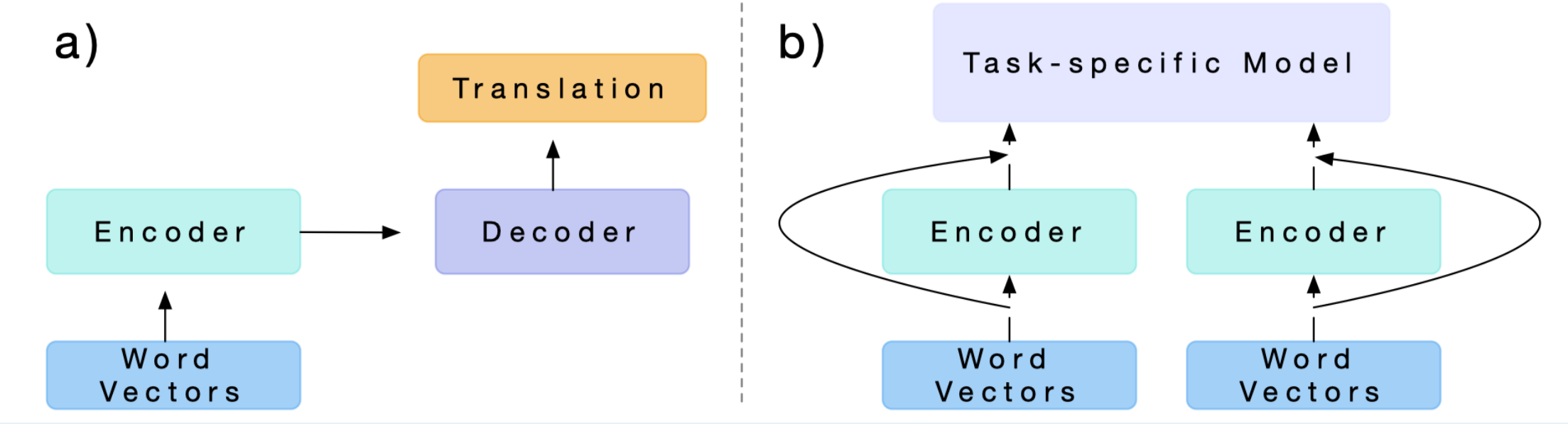
**双语线索** 不同译词帮助区分源语言词义

**局限** 义项数量与划分依赖数据，仍需决定当前使用哪个义项

Guo等，COLING 2014。词义特定表示是上下文词向量发展中的早期探索。

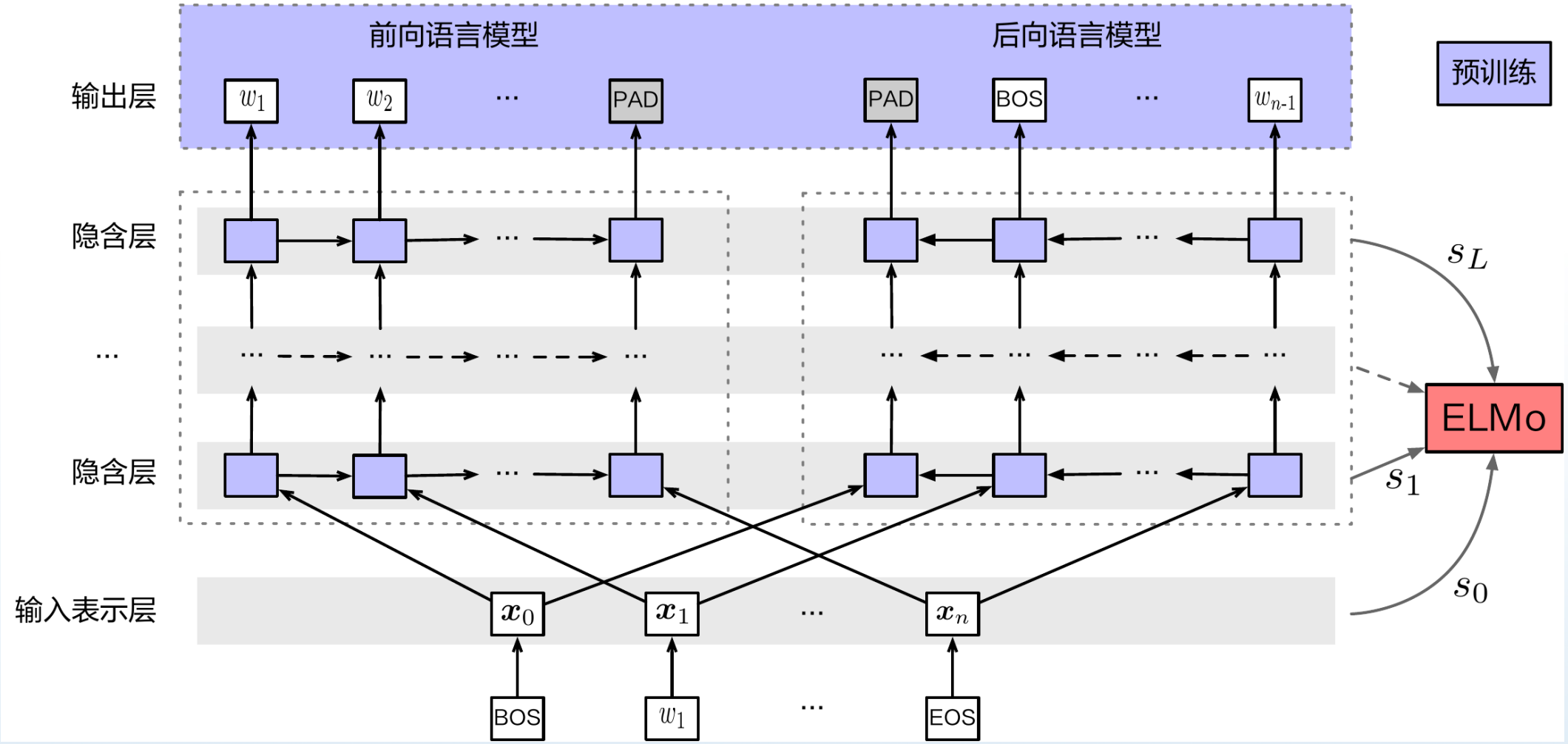
# CoVe：迁移机器翻译编码器

## 2017年：先训练机器翻译，再复用编码器的上下文表示。



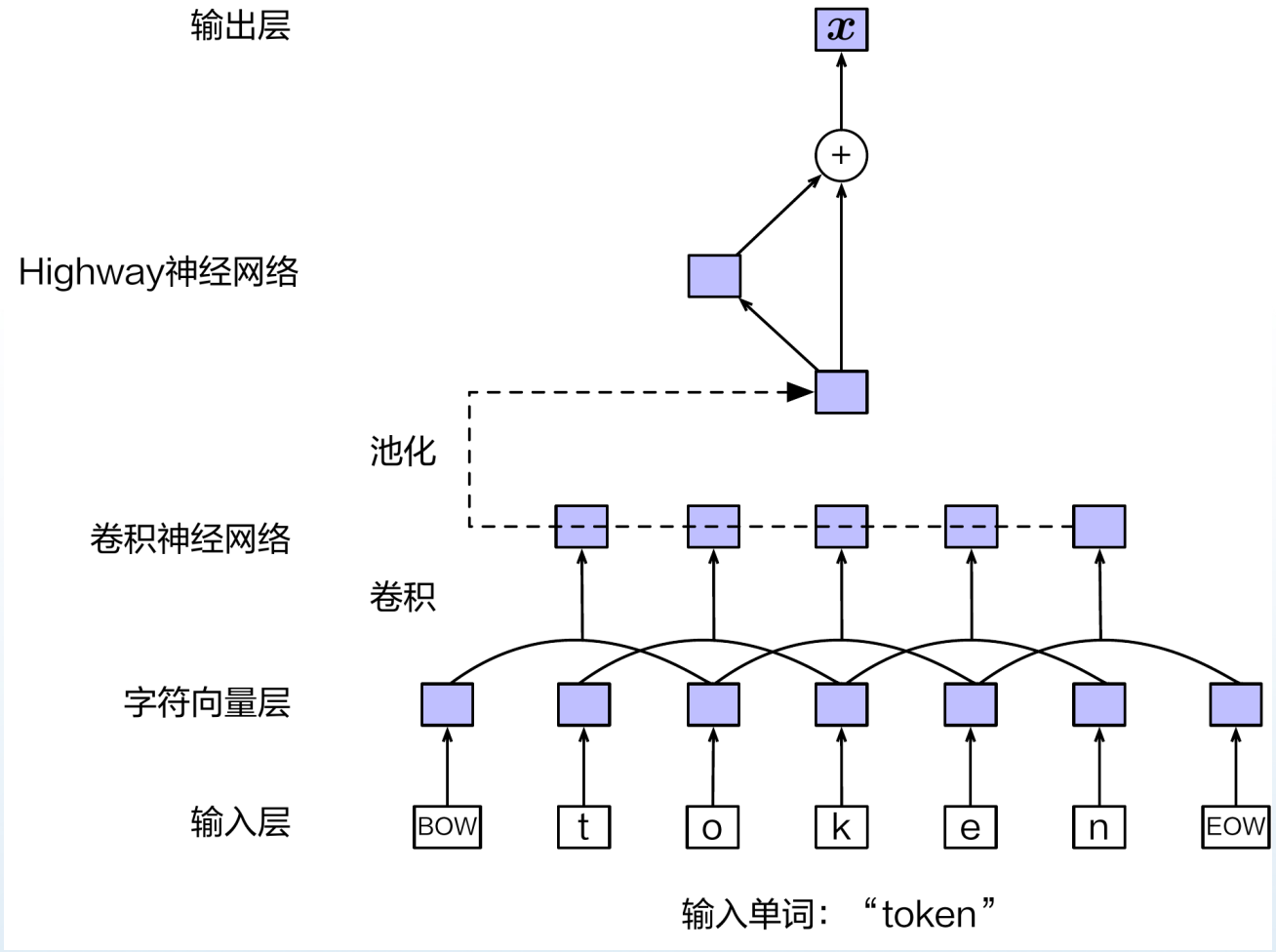

左：翻译训练。右：下游迁移。CoVe需要平行语料；同期TagLM改用单语语言模型特征，随后发展为ELMo。

# ELMo：深层双向语言模型表示




ELMo，NAACL 2018。字符词形编码、双向语言模型与任务层融合。

# 字符CNN：先根据词形构造表示




输入：[B, T, C]
C为每词字符槽位数

卷积提取词内片段
池化汇总局部特征

此时尚未融合句子上下文

字符CNN产生词形表示。随后BiLM才把当前位置与句子上下文联系起来。

# 双向语言模型：两个方向分别预测

| 位置 | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 输入 | BOS | 我 | 喜欢 | 读书 | EOS |
| 前向目标 | 我 | 喜欢 | 读书 | EOS | — |
| 后向目标 | — | BOS | 我 | 喜欢 | 读书 |

**前向只看左边，后向只看右边，两项损失相加。**

**共享参数** 字符CNN输入表示层与Softmax输出层由两个方向共享

**独立参数** 前向LSTM与后向LSTM各自堆叠，互不读取对方的隐状态

两条方向各自堆叠LSTM。表中的“—”不参与损失，每个方向有4个有效目标。

# ELMo多层表示的任务加权

$$ELMo_t = \gamma \Sigma_j s_j h_{t,j}，\ s = softmax(a)$$

| 表示 | 包含的信息 | 融合前形状 |
| --- | --- | --- |
| $h_0$ | 字符CNN的词形表示 | [B, T, D] |
| $h_1$、$h_2$ | 不同深度的双向上下文 | 各为[B, T, D] |

$h_0$与上下文层先统一维度。s是任务级权重，γ调节整体尺度，均可学习。

# PyTorch实现：三层表示的融合

```
# h0、h1、h2均为[B, T, D]，已统一维度
a = nn.Parameter(torch.zeros(3))
gamma = nn.Parameter(torch.tensor(1.0))
layers = torch.stack([h0, h1, h2], dim=0)
s = a.softmax(dim=0)
weights = s[:, None, None, None]
elmo = gamma * (weights * layers).sum(dim=0)
# 初始化时s为[1/3, 1/3, 1/3]
```

冻结编码器后，仍需让a、γ和任务模型接收梯度。参数只在模型初始化时创建。

# ELMo特征在下游任务中的接入




**任选一处或两处拼接；教材ELMoMLP示例则直接用$ELMo_k$替换静态词向量。**

| 下游任务 | 更有效的ELMo层（Peters等，2018） |
|---|---|
| 词性标注 | 第1层LSTM优于第2层：低层更偏词法、句法 |
| 词义消歧 | 第2层LSTM明显优于第1层：高层更偏语义 |

编码器冻结时，只训练层权重s、缩放γ和任务模型。词元任务保留每个位置，句子任务再做池化。

# 教材示例：GloVe与ELMo的近邻对比

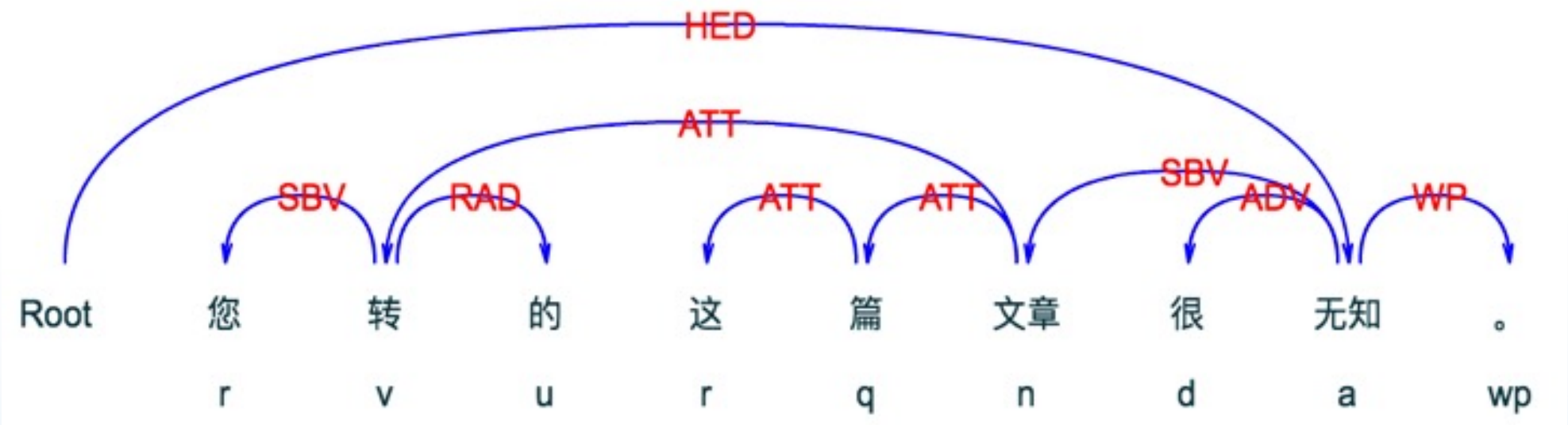
| 模型 | 查询 | 最近邻 |
| --- | --- | --- |
| GloVe | play | playing, game, games, played, players, plays, player, Play, football, multiplayer |
| ELMo | Chico Ruiz made a spectacular **play** on Alusik's grounder ...<br>义项：比赛中的表现 | Kieffer ... was commended for his ability to hit in the clutch, as well as his all-round excellent **play** |
| ELMo | Olivia De Havilland signed to do a Broadway **play** for Garson ...<br>义项：戏剧 | ... they were actors who had been handed fat roles in a successful **play**, and had talent enough to fill the roles competently ... |

**GloVe的近邻混合多种词性与用法；ELMo找到的是用法一致的句中位置。**

**启示**　　上下文向量具备一定的词义消歧能力，可以按“句中位置”而不是“词类型”做检索

Peters等，NAACL 2018，教材“上下文相关的词义相似性检索”表。英文例句有删节。

# 历史案例：ELMo与依存句法分析




| CoNLL 2018系统 | 官方LAS |
| --- | --- |
| HIT-SCIR（第1名） | 75.84 |
| TurkuNLP（第2名） | 73.28 |

2018年历史结果。与第2名相差2.56分，包含整套系统差异，不能全部归因于ELMo。

# 词向量、上下文词向量与句向量

| 表示 | 输入 | 典型输出与用途 |
| --- | --- | --- |
| 静态词向量 | 词ID或词形 | 一个词向量，快速查询 |
| 上下文词向量 | 句子与词元位置 | 各位置向量，标注与理解 |
| 句子 / 文档嵌入 | 整段文本 | 整段向量，相似检索 |

**评价与训练目标，应与最终使用的表示粒度一致。**

相同的“Embedding”名称可能指不同粒度。需要先看输入、输出和训练目标。

# 研究补充：从字符片段到字节级模型

| 2016 | 2017 | 2018 | 2024 | 2025 |
|---|---|---|---|---|
| **子词BPE**<br>高频字符片段逐步合并成子词表<br>Sennrich等，ACL 2016 | **fastText**<br>词向量加上字符n-gram向量<br>Bojanowski等，TACL 2017 | **ELMo**<br>字符CNN构造词形表示，再接BiLM<br>Peters等，NAACL 2018 | **BLT**<br>按下一字节的熵动态切分字节块<br>Pagnoni等，ACL 2025 | **H-Net / Bolmo**<br>端到端学习分块；子词模型字节化<br>Hwang等；Minixhofer等 |

**共同问题是未登录词：粒度从词、子词走向字节，切分也开始由模型学习。**

字节级模型仍在快速发展，效率与效果需按语言和任务验证。中文没有空格，分词与切分问题更突出。

# 研究补充：面向检索的文本与多模态嵌入

| 模型 | 输入对象 | 特点 |
| --- | --- | --- |
| Qwen3 Embedding，2025 | 多语言文本与代码 | 基于大模型，配套重排序 |
| EmbeddingGemma，2025 | 多语言文本 | 约3亿参数，面向端侧 |
| Qwen3-VL-Embedding，2026 | 文本、图像、文档图像、视频 | 统一跨模态空间 |
| Gemini Embedding 2，2026 | 文本、图像、视频、音频、PDF | 原生多模态输入 |

**向量检索** 查询与候选分别编码，用相似度快速选出候选

**重排序** 模型联合读取查询与候选，对候选进一步评分

**可变维度** Matryoshka式训练使截短后的向量仍可用，在存储与精度间折中

表示对象已从词扩展到句子、文档和多模态内容。相似度检索仍沿用本章的余弦与近邻思想。

# 研究补充：大模型中的词向量层




| 观察 | 与本章的联系 |
| --- | --- |
| 输入仍是查表 | 同一词元ID得到同一输入向量，上下文信息由后续层加入 |
| 规模更大 | 词表常超过10万（如Llama 3约12.8万），D达到数千维 |
| 线性表示 | 许多概念对应表示空间中的方向，延续了词类比中的“关系方向” |

Press & Wolf，EACL 2017（权重绑定）；Park等，ICML 2024（线性表示假设）。下一章展开预训练语言模型。

# 本章小结

| | |
|---|---|
| **学习来源** | 共现统计和文本预测提供自监督信号 |
| **静态表示** | CBOW、SGNS和GloVe为词类型学习向量 |
| **动态表示** | ELMo根据上下文生成词元表示，并融合多层信息 |
| **迁移评价** | 按任务检验预训练收益 |

# 谢 谢!

理解语言，认知社会

以中文技术，助民族复兴



长按二维码，关注哈工大SCIR

微信号：HIT_SCIR